CSE 242 Assignment 2

3 Questions, 100 pts

Your name: Chirag Manish Bolakani Student ID: 2346802

Instruction

- Submit your assignments onto **Canvas** by the due date. Upload a `zip` file containing:

  (1) The saved/latest `.ipynb` file.

  (2) Also save your file into a pdf version, if error appears, save an html version instead (easy to grade for written questions).

  (3) All other materials to make your `.ipynb` file runnable.

  For assignment related questions, please reach TA or grader through EdDiscussion.
- This is an **individual** assignment. All help from others (from the web, books other than text, genAI, or people other than the TA or instructor) must be clearly acknowledged.
- Most coding parts can be finished with only 1-2 lines of codes.
- Make sure you have installed required packages: `scikit-learn`.
- Double click to edit each markdown cell.

Objective

- **Task 1:** Maximum likelihood estiamtion (math)
- **Task 2:** Linear Regression (with Scikit-learn)
- **Task 3:** Principle Component Analysis (with Scikit-learn)

Question 1 (Maximum likelihood estiamtion and Posterior, 20 pts)

Assume we have a coin that has some unknown probability h of coming up heads (and probability $1-h$ of coming up tails). If the coin is flipped five times getting three heads and two tails (HHHTT) then:

(a -- 10pts) What is the maximum likelihood estimate for h?

(b -- 10pts) Assume that (before flipping the coin) we have a prior density $p(h)$ for the various values of $h \in [0,1]$. Give the formula for the posterior probability density $p(h \mid \mathrm{HHHTT})$ as a function of the prior $p(h)$.

For Question 1(b), the solution is acceptable if the formula contains an integral.

Solution (a):

Each flip is independent, hence the probability of the whole sequence is the product of the individual flip probabilities.

We know that Heads has probability h and tails has probability $1-h$

The flips are independent, so the likelihood of observing HHHTT is

$$L(h) = p(\mathrm{HHHTT} \mid h) = h^3(1-h)^2$$

By taking the logarithm, we get:

$$\ell(h) = 3\ln h + 2\ln(1-h)$$

At a maximum of a smooth function, the slope is zero. Set the derivate to 0 we get:

Maximize the log-likelihood $\ell(h) = 3\ln h + 2\ln(1-h)$:

$$\frac{d\ell}{dh} = \frac{3}{h} - \frac{2}{1-h} = 0$$

$$3(1-h) = 2h$$

$$\hat{h}_{MLE} = \frac{3}{5} = 0.6$$

We also confirm its maximum and not minimum by taking the second derivative:

$$\frac{d^2\ell}{dh^2} = -\frac{3}{h^2} - \frac{2}{(1-h)^2}$$

Since

$$-\frac{3}{h^2} - \frac{2}{(1-h)^2} < 0$$

Both the terms are negative i.e. the function curves downward

Therefore, it is at maximum.

Solution (b):

Using Bayes' theorem:

$$p(h \mid \text{HHHTT}) = \frac{p(\text{HHHTT} \mid h)p(h)}{p(\text{HHHTT})}$$

the likelihood for 3 heads and 2 tails:

$$p(\text{HHHTT} \mid h) = h^3(1-h)^2$$

The denominator is obtained by integrating over all possible values of h. Therefore, the posterior probability density is:

$$p(h \mid \text{HHHTT}) = \frac{p(\text{HHHTT} \mid h)\, p(h)}{p(\text{HHHTT})}$$
$$= p\frac{h^3(1-h)^2\, p(h)}{\displaystyle\int_0^1 t^3(1-t)^2\, p(t)\, dt}, \qquad h \in [0,1].$$

If you are not familair with Latex, you may attach a figure/screen-shoot and display the code below.

```
from IPython.display import Image
# Replace the figure name
# Image(filename='sample.png')
```

Question 2 (Linear Regression, 40 pts)

In this question, you will be using **Scikit-learn** to empirically apply the Linear Regression model. We will adopt an advertisement dataset **Advertising.csv** from Kaggle.

Reading data using Pandas

```
# Read the dataset you will be working on
# The dataframe loaded with pandas is named as data

import pandas as pd
data = pd.read_csv('Advertising.csv', index_col=0)  # Modify your data path accordingly.

# Take a look at the first 5 rows
data.head()
```

	TV	Radio	Newspaper	Sales
1	230.1	37.8	69.2	22.1
2	44.5	39.3	45.1	10.4
3	17.2	45.9	69.3	9.3
4	151.5	41.3	58.5	18.5
5	180.8	10.8	58.4	12.9

Next steps: Generate code with data | New interactive sheet

```
data.shape
```

```
(200, 4)
```

Features and responses

What are the features?

- **TV:** advertising dollars spent on TV for a single product in a given market (in thousands of dollars)
- **Radio:** advertising dollars spent on Radio
- **Newspaper:** advertising dollars spent on Newspaper

What is the response?

- **Sales:** sales of a single product in a given market (in thousands of items)

What else do we know?

- There are 200 **observations**, and each observation (each row) is a single market.
- **Your main task**: predict the sales of a single product in a given market.
- Since the response variable is continuous, this is a **regression** problem.

Linear regression

Pros: fast, no tuning required, highly interpretable

Cons: adopting the linear regression is unlikely to always generate the best predictive accuracy, since Linear Regression presumes a linear relationship between the features and response. This assumption may not always make sense. (But if one can *transform* the features so that do have a linear relationship, sometimes progress can be made, for example by taking logs.)

Expression:

- y: the response
- β_0: the intercept
- β_1: the coefficient for x_1 (the first feature)
- β_i is the coefficient for x_i (the ith feature, $i \in \{1, 2, \ldots, n\}$)

For this regression task:

$$\text{Sales} = \beta_0 + \beta_1 \times \text{TV} + \beta_2 \times \text{Radio} + \beta_3 \times \text{Newspaper}$$

The β values ($\beta = [\beta_0, \beta_1, \beta_2, \beta_3]$) are called the **model coefficients**. These values are "learned" during the model fitting step using the "least squares" criterion. Then, we can make use of the fitted model to make predictions!

Prepare the dataset for training use

- Scikit-learn expects X (feature matrix) and y (response vector) to be NumPy arrays.
- Pandas is built on top of NumPy, and exposes numpy methods. Thus, X can be a pandas DataFrame, y can be a pandas Series!

```
# Prepare the feature X
X = data[['TV', 'Radio', 'Newspaper']]

# check the type and shape of X
print(type(X))
print(X.shape)
```

```
<class 'pandas.core.frame.DataFrame'>
(200, 3)
```

```
# Prepare the response Y
y = data['Sales']
# check the type and shape of y
print(type(y))
print(y.shape)
```

```
<class 'pandas.core.series.Series'>
(200,)
```

Question 2.1 (Split the dataset into train and test parts, 5 pts)

For features X and response y, use **sklearn** to perform the the splitting of dataset: 80% for train data, 20% for the test data.

```
################# Your answer for Question 2.1   #################
from sklearn.model_selection import train_test_split
# test_size 0.2 splits the dataset into 20% testing and 80% training
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
################# Your code above #################
```

Double check the shape of train and test data:

```
# default split is 80% for training and 20% for testing
print(X_train.shape)
print(y_train.shape)
print(X_test.shape)
print(y_test.shape)
```

```
(160, 3)
(160,)
(40, 3)
(40,)
```

Question 2.2 (Train a Linear regression model via Scikit-learn, 10 pts)

Assume you model is named as **model**, and you train the linear regression model to fit on the training data.

```
################# Your answer for Question 2.2   #################
# Name your linear regression model as 'model', so proceeding cells won't run into errors.

from sklearn.linear_model import LinearRegression

# create the Linear Regression model
model = LinearRegression()
# use only training data so test data remains unseen
model.fit(X_train, y_train)

################# Your code above #################
```

```
▾ LinearRegression
LinearRegression()
```

```
# Take a look at your model coefficients
import numpy as np
print(np.round(model.intercept_, 4))
print([np.round(val, 4) for val in model.coef_])
```

```
2.9791
[np.float64(0.0447), np.float64(0.1892), np.float64(0.0028)]
```

```
# Match the feature names with the trained coefficients
list(zip(data.columns[:3].tolist(), model.coef_))
```

```
[('TV', np.float64(0.044729517468716326)),
 ('Radio', np.float64(0.18919505423437652)),
 ('Newspaper', np.float64(0.0027611143413671935))]
```

Sales = 2.9791 + 0.0447·TV + 0.1892·Radio + 0.0028·Newspaper

Question 2.3 (Interpreting model coefficients, 10 pts)

The coefficients of each feature are printed above.

Your tasks:

- What is the trained linear regression model?

- **(replace β_i below with the trained coefficients, reserve four decimal places, 5 pts)**

$$\text{Sales} = \beta_0 + \beta_1 \times \text{TV} + \beta_2 \times \text{Radio} + \beta_3 \times \text{Newspaper}$$

- **(Open) question (5 pts)**: how do you interpret the coefficient of **TV** (β_1)?

Your solution for Question 2.3:

Trained model:

$$\text{Sales} = 2.9791 + 0.0447 \times \text{TV} + 0.1892 \times \text{Radio} + 0.0028 \times \text{Newspaper}$$

Interpretation of β_1 (TV = 0.0447):

Holding Radio and Newspaper advertising spending constant, each additional **$1,000 spent on TV advertising** is associated with an increase of approximately **0.0447 thousand units** in Sales (44.7 units).

Equivalently, an additional **$100,000 spent on TV advertising** is associated with approximately **4,470 more units sold**.

TV has the largest coefficient, suggesting a stronger effect per unit of spending. However, this alone does not establish that TV accounts for the largest proportion of variation in Sales.

Making predictions

```
# make predictions on the testing set
y_pred = model.predict(X_test)
# Print the test accuracy score from your trained linear regression model
model.score(X_test,y_test)
```

```
0.899438024100912
```

Model evaluation metrics for regression

Evaluation metrics for classification problems, such as **accuracy**, are not useful for regression problems. Instead, we need evaluation metrics designed for comparing continuous values. (Since predictions would hardly ever exactly match the measured quantities.)

Mean Squared Error (MSE) is the mean of the squared errors:

$$\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Root Mean Squared Error (RMSE) is a popular evaluation metric for regression, which is defined as the square root of the mean of the squared errors:

$$\sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

Question 2.4 (Evaluate your predictions on test features with RMSE metric, 5 pts)

Hint: you may calculate MSE using scikit-learn, but RMSE is a little bit different from MSE.

```
from sklearn import metrics
################# Your answer for Question 2.4   #################

mse = metrics.mean_squared_error(y_test, y_pred)
print(f"Test MSE (TV, Radio, Newspaper): {np.round(mse, 4)}")

# RMSE is the square root of the mean of the squared errors i.e MSE
rmse = np.sqrt(mse)
print(f"Test RMSE (TV, Radio, Newspaper): = {np.round(rmse, 4)}")
################# Your code above #################
```

```
Test MSE (TV, Radio, Newspaper): 3.1741
Test RMSE (TV, Radio, Newspaper): = 1.7816
```

Question 2.5 (Does the feature 'Newspaper' help? 10 pts)

Does the feature **Newspaper** contribute or impair the model prediction? Now repeat previous procedure:

- For features X, only select **TV** and **Radio**; response y is unchanged.
- Split the dataset as what you have done before.
- Fit/Train the model on the train data.
- Make predictions on the test data.
- Compute the RMSE of the predictions on the test data.
- **Show your observations!**

Reminder: if you use `random_state` while splitting the dataset, make sure the value of `random_state` is the same for the two splits. So that the test data only differs in the column of **Newspaper**.

```
################# Your answer for Question 2.5   #################

# only use TV and radio
X2 = data[['TV', 'Radio']]
# split train and test
X2_train, X2_test, y2_train, y2_test = train_test_split(X2, y, test_size=0.2, random_state=42)
# train the model
model2 = LinearRegression().fit(X2_train, y2_train)
# predict with test data
y2_pred = model2.predict(X2_test)
rmse2 = np.sqrt(metrics.mean_squared_error(y2_test, y2_pred))
print(f"Coefficients (TV, Radio): {np.round(model2.coef_, 4)}  intercept: {model2.intercept_:.4f}")
print(f"Test RMSE with Newspaper   : {rmse:.4f}")
print(f"Test RMSE without Newspaper: {rmse2:.4f}")
################# Your code above #################
```

```
Coefficients (TV, Radio): [0.0447 0.1907]  intercept: 3.0283
Test RMSE with Newspaper   : 1.7816
Test RMSE without Newspaper: 1.7714
```

Your observations:

Removing **Newspaper** slightly *lowers* the test RMSE from 1.7816 to 1.7714, so Newspaper does not help much and in fact marginally impairs prediction on this test set.

In a model trained with all three features, its coefficient is tiny (0.0028) compared to TV (0.0447). If trained with just TV and radio, the coefficients barely change (0.0447 for TV and 0.1907 for Radio). Hence, Newspaper carries very little information about sales. Including newspaper simplt adds an extra parameter without contributing much to the sales prediction.

Question 3 (PCA, 40 pts)

In this question, we will explore how PCA helps with reducing the dimensionality through scikit-learn. We will adopt a simple digits dataset contained in scikit-learn.

```
# import libraries
%matplotlib inline
import matplotlib.pyplot as plt
import seaborn as sns;
sns.set()
```

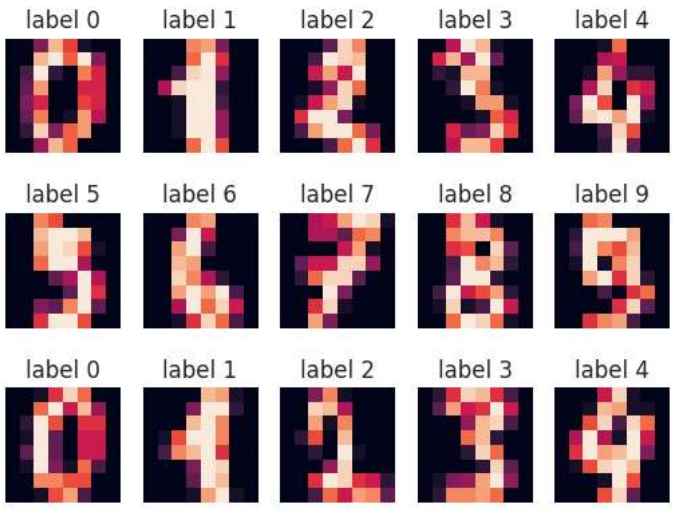
```
from sklearn.datasets import load_digits
data_digit = load_digits()
data_digit.data.shape
```

```
(1797, 64)
```

As you have seen above, there are 1797 rows and 64 columns in this dataset. Each row denotes the pixel values of an image. Since each image is 8*8 (in pixel), the dimension of each feature is 64. Let's take a look at a few images contained in this dataset.

```
# A help function to display images with labels
def display_digits(imgs):
    h = 3
    w = 5
```

```
    fig, axes = plt.subplots(h, w)
    for i in range(h):
        for j in range(w):
            if imgs.shape[1] == 64:
                axes[i, j].imshow(imgs[i * w + j].reshape(8, 8))
            else:
                axes[i, j].imshow(imgs[i * w + j])
            axes[i, j].axis('off')
            axes[i, j].set_title(f'label {data_digit.target[i * w + j]}')
# Take a look at a few examples
display_digits(data_digit.images)
```



Double-click (or enter) to edit

PCA finds uncorrelated orthogonal axes (principal components) in a high dimensional space (i.e., 64) to project the feature onto principal components.

Question 3.1 (PCA and dimensionality deduction with Scikit-learn, 10 pts)

In this question, you may use the implementation of PCA from scikit-learn, as imported below. Detailed explanations of parameters settings are available in this link.

Your task in this question:

Fit the model with feature X (**data_digit.data** for this dataset) and apply the dimensionality reduction on X, keep 2 components.

```
from sklearn.decomposition import PCA

################## Your answer for Question 3.1   ##################
# pca with 2 components
pca_2 = PCA(n_components=2)
# name the transformed (dimension reduced) feature as projected
projected = pca_2.fit_transform(data_digit.data)
################## Complete the code above ##################


# The code below checks whether the feature dimension 64 is reduced to 2.
print(data_digit.data.shape)
print(projected.shape)
```

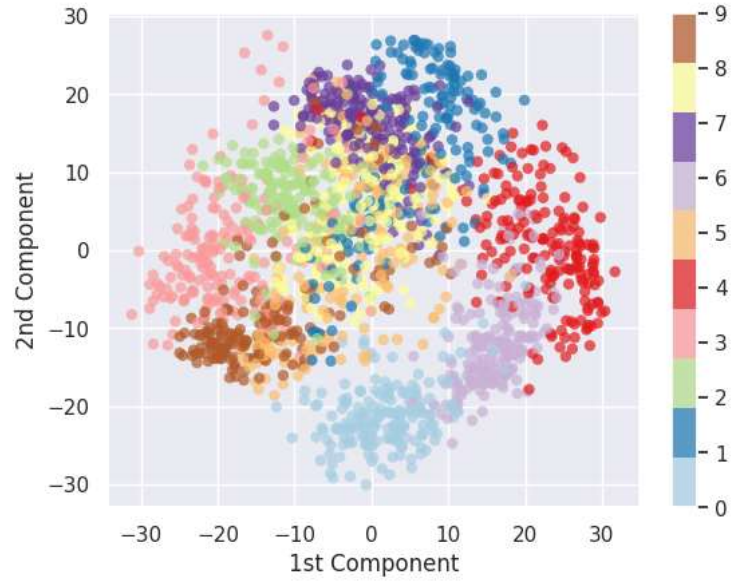
```
(1797, 64)
(1797, 2)
```

With the reduced dimension of features, we can now visualize the projected/reduced features in a 2-dimension plot as below. There are hardly any overlaps between digit 0 and any other digits. While in the central area of the figure, features are not well separable.

```
plt.scatter(projected[:, 0], projected[:, 1],
            c=data_digit.target, edgecolor='none', alpha=0.7,
            cmap=plt.get_cmap('Paired', 10))
```

```
plt.xlabel('1st Component')
plt.ylabel('2nd Component')
plt.colorbar()
plt.figure(figsize=(18, 8))
```

```
<Figure size 1800x800 with 0 Axes>
```




```
<Figure size 1800x800 with 0 Axes>
```

Question 3.2 (How many components do we need? 15 pts)

In practice, suppose you are given a high-dimension dataset, and you are interested in performing a dimension reduciton with PCA. How many conponents do we need? One method is to look at the distribution of the explained variance ratio w.r.t. each component.

In this question, you may use the function `explained_variance_ratio` in **pca** (with scikit-learn) to address this question. Detailed explanations of parameters settings are available in this link.

Your task in this question:

Print the ratio of variance explained by each of the selected components;

Visualize with matplotlib or seaborn (x axis: $i-th$ component; y axis: the corresponding ratio of explained variance).

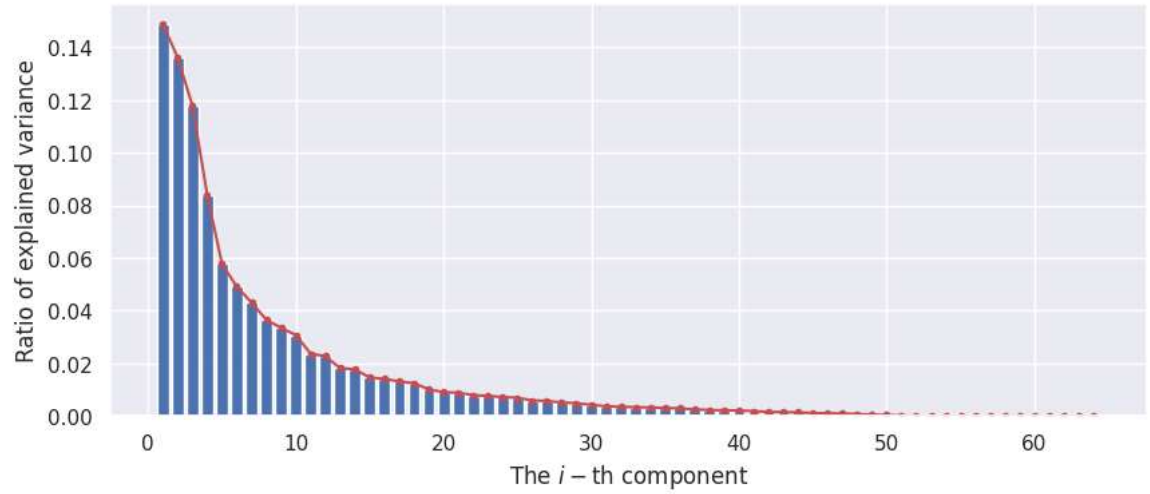
```
################## Your answer for Question 3.2   ##################

# fit pca with all the components
pca_all = PCA().fit(data_digit.data)

# explained_variance_ratio_ attribute represents the percentage of the dataset's total variance that is captured by each indivi
ratios = pca_all.explained_variance_ratio_
print(np.round(ratios, 4))
print('Cumulative:', np.round(np.cumsum(ratios), 4))

# plot each component's explained variance ratio
plt.figure(figsize=(10, 4))
plt.bar(np.arange(1, len(ratios) + 1), ratios)
plt.plot(np.arange(1, len(ratios) + 1), ratios, 'r.-')
plt.xlabel('The $i-$th component')
plt.ylabel('Ratio of explained variance')
################## Complete the code above ##################
```

```
[1.489e-01 1.362e-01 1.179e-01 8.410e-02 5.780e-02 4.920e-02 4.320e-02
 3.660e-02 3.350e-02 3.080e-02 2.370e-02 2.270e-02 1.820e-02 1.770e-02
 1.470e-02 1.410e-02 1.320e-02 1.250e-02 1.020e-02 9.100e-03 8.900e-03
 8.000e-03 7.700e-03 7.200e-03 7.000e-03 6.000e-03 5.800e-03 5.200e-03
 4.900e-03 4.300e-03 3.700e-03 3.500e-03 3.400e-03 3.300e-03 3.100e-03
 2.900e-03 2.600e-03 2.300e-03 2.200e-03 2.100e-03 1.900e-03 1.600e-03
 1.500e-03 1.400e-03 1.200e-03 1.100e-03 1.000e-03 8.000e-04 6.000e-04
 4.000e-04 2.000e-04 1.000e-04 1.000e-04 1.000e-04 0.000e+00 0.000e+00
 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00
 0.000e+00]
Cumulative: [0.1489 0.2851 0.403  0.4871 0.545  0.5941 0.6373 0.6739 0.7074 0.7382
 0.762  0.7847 0.8029 0.8206 0.8353 0.8494 0.8626 0.8751 0.8852 0.8943
 0.9032 0.9112 0.9188 0.9261 0.933  0.939  0.9447 0.9499 0.9548 0.9591
 0.9628 0.9664 0.9697 0.973  0.9761 0.979  0.9816 0.9839 0.9861 0.9882
 0.9901 0.9917 0.9932 0.9946 0.9958 0.9968 0.9978 0.9986 0.9991 0.9995
 0.9998 0.9998 0.9999 0.9999 1.     1.     1.     1.     1.     1.
 1.     1.     1.     1.     ]
Text(0, 0.5, 'Ratio of explained variance')
```



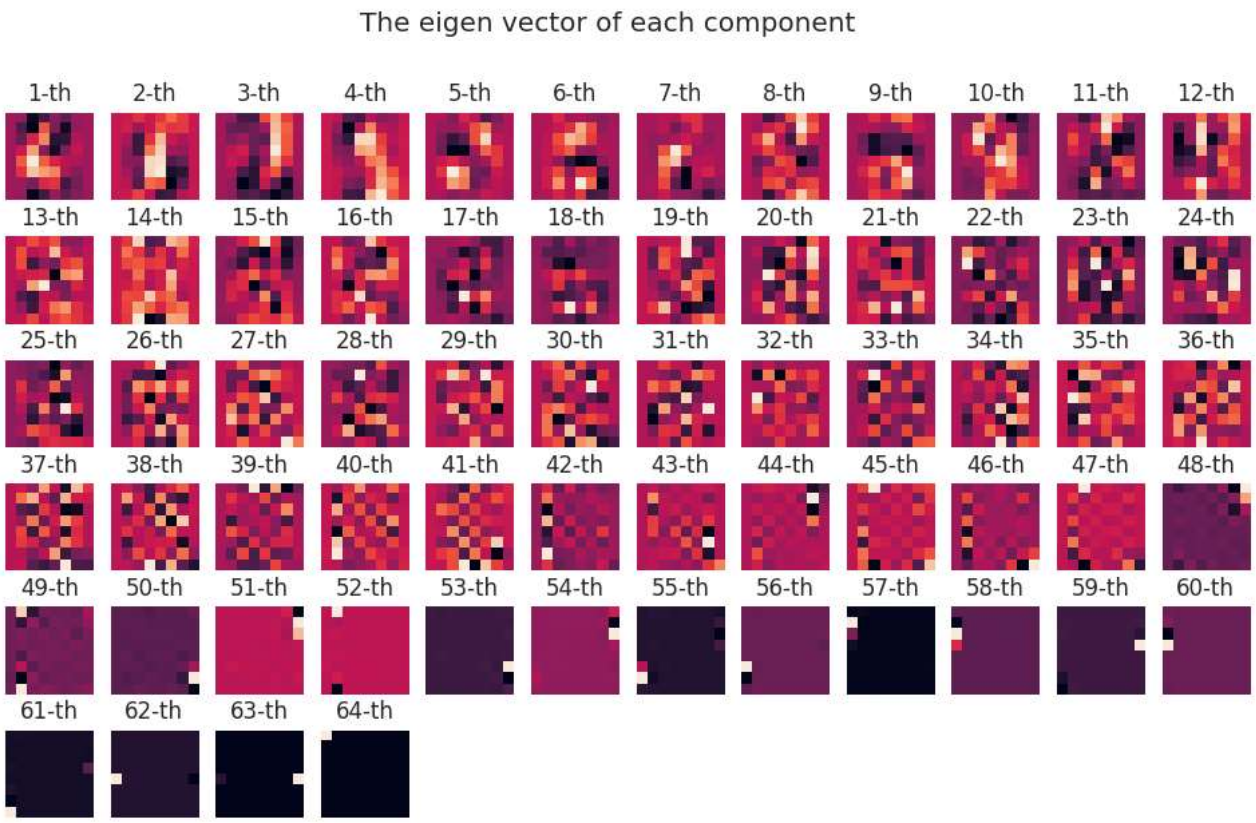
How to obtain eigen values and vectors from sklearn PCA

- Eigen values -- `your-PCA-model.explained_variance_`
- Eigen vectors -- `your-PCA-model.components_`

Eigen vectors (Eigen digits)

```
# Default # of components is the dimension of feature space
pca = PCA()
pca.fit(data_digit.data)
h = 6
w = 12
fig, axes = plt.subplots(h, w)
fig.set_size_inches(12, 7)
for i in range(h):
    for j in range(w):
        idx = i * w + j
        if idx > 63:
            axes[i, j].axis('off')
        else:
            axes[i, j].imshow(pca.components_[idx].reshape(8, 8))
            axes[i, j].axis('off')
            axes[i, j].set_title(f'{idx + 1}-th')
fig.suptitle('The eigen vector of each component')
```

```
Text(0.5, 0.98, 'The eigen vector of each component')
```



Question 3.3 What are your observations from the visualized eigen vectors? (5 pts)

Your solutions for Question 3.3:

Initial components show digit representation. The first few eigenvectors (1st to 10th) show digit like representation. They have loops, diagonal lines and patterns that match digits like 7, 3, 2, etc. **Later components have a lot of noise.** Moving to higher indices the patterns become noisier and less interpretable. They contribute much less variations. **The last components are almost blank.** After approximately 48th component the images look almost random containing no meaningful patterns. Some of the images are almost pitch black and blank. Hence, we can conclude that most of the meaningful information is present in the first 30 components.

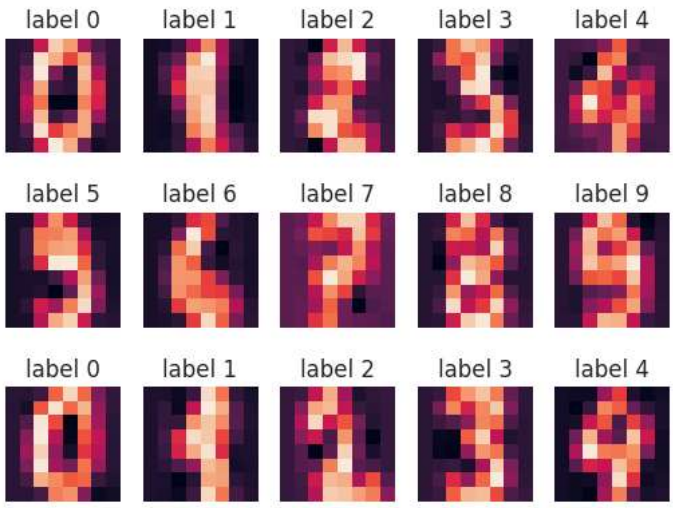
Feature reconstruction:

Suppose you only want to preserve 80% variance after applying PCA for the dimension reduction, due to storage issues, etc.

```
# Use pca from sklearn to fit on the feature space
pca_08 = PCA(0.8).fit(data_digit.data)

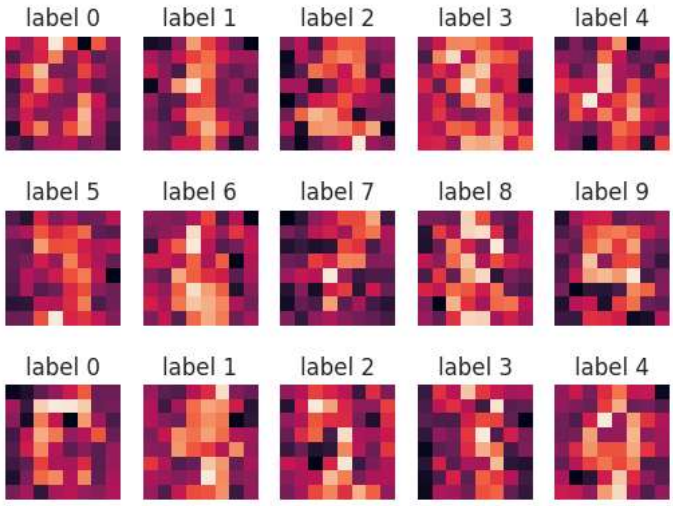
# Reconstruct
lower_dimension = pca_08.fit_transform(data_digit.data)
reconstructed = pca_08.inverse_transform(lower_dimension)

# Display the reconstructed images (with a much lower dimension)
display_digits(reconstructed)
```



Assume now you are given only a set of perturbed images (with some random noise). As shown below, the figures become much more difficult to recognize.

```
# Apply random noise on clean images
noisy_imgs = np.random.normal(data_digit.data, 5)
# Display the noisy images
display_digits(noisy_imgs)
```



Question 3.4 Does PCA mitigate the random noise? (10 pts)

Your tasks:

- Perform feature reconstruction on `noisy_imgs` (preserving 80% variance after applying PCA for the dimension reduction)
- Explain your understandings on why PCA reconstructs cleaner images.

```
################## Your answer for Question 3.4   ##################

# preserving 80% variance
pca_noise = PCA(0.8).fit(noisy_imgs)
print(f"Components kept for 80% variance on noisy data = {pca_noise.n_components_}")
# name the reconstructed images from noisy images as 'reconstructed_noise'
reconstructed_noise = pca_noise.inverse_transform(pca_noise.transform(noisy_imgs))
# Display the reconstructed images
display_digits(reconstructed_noise)
################## Complete the code above ##################
```

```
Components kept for 80% variance on noisy data = 41
```

